House Price Prediction System

Project Id: PY-PT48

Name : Ketan Agarwal (Team Leader)

Class Roll Number: 48

University Roll Number : 2028742

Section and Group: P G2

Name : Harsh Jain

Class Roll Number: 45

University Roll Number : 2028721

Section and Group: P G2

Name : Yaduveer Yadav

Class Roll Number: 79

University Roll Number : 2028911

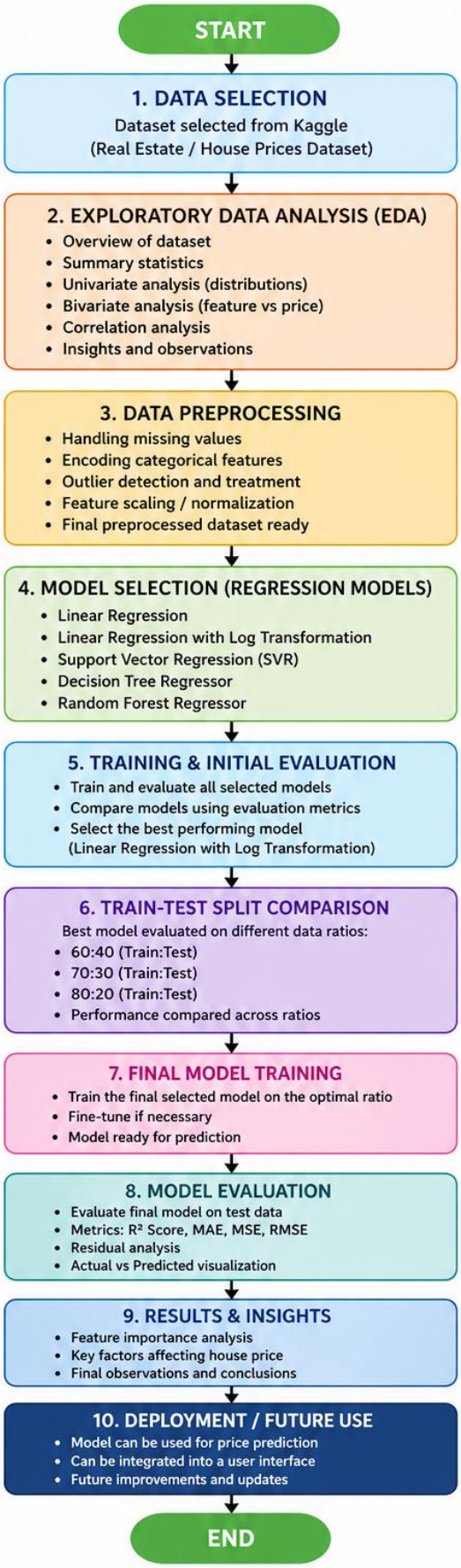
Section and Group: P G2

1. **Problem Description** 3
2. **Architecture** 4
3. **Models model** 5-6
4. **Evaluation metrics used** 7-8
5. **Preliminary Results and Visualizations** 9-11
6. **References** 12

1. Problem Description

The House Price Prediction System is a Python-based machine learning application developed to automate the analysis and prediction of residential property prices. It streamlines the process of handling real-estate data, training predictive models, and generating accurate price estimations while maintaining efficiency and reliability. Using core concepts of Machine Learning such as data preprocessing, feature engineering, regression algorithms, model evaluation, and data visualization techniques, this project reduces manual effort in property valuation and supports data-driven decision-making in the real estate domain.

2. Architecture



3. Models Used (Applied/ Planned):

- In this project, multiple regression models were implemented and evaluated to predict house sale prices based on structured real-estate data. The use of diverse models allowed for comparative analysis and helped in identifying the most suitable approach for capturing relationships between property features and pricing.
- The models applied include Linear Regression, Linear Regression with Log Transformation, Support Vector Regression (SVR), Decision Tree Regressor, and Random Forest Regressor.

1. Linear Regression (Baseline Model)

- Linear Regression was initially implemented as a baseline model to establish a reference for performance comparison. It is a simple yet powerful statistical method that assumes a linear relationship between independent variables (features) and the dependent variable (house price).
- The model was trained using the original dataset without any transformation applied to the target variable. This approach provides a straightforward interpretation of how each feature contributes to the predicted price.
- However, due to the nature of real estate data, which often exhibits skewness and non-linearity, the performance of the basic linear regression model may be limited. Despite this, it serves as an essential benchmark for evaluating more advanced models.

2. Linear Regression with Log Transformation (Final Selected Model)

- To address the issue of skewness in the target variable (SalePrice), a logarithmic transformation was applied using log1p. This transformation helps stabilize variance and makes the data distribution more normal, which improves the performance of linear models.
- The model was trained on the transformed target variable, and predictions were later converted back to the original scale using the inverse transformation (expm1). This approach significantly enhances prediction accuracy and reduces the impact of extreme values (outliers).
- This model was ultimately selected as the final model due to its improved performance metrics and better generalization ability compared to other models. It effectively balances simplicity, interpretability, and accuracy.

3. Support Vector Regression (SVR)

- Support Vector Regression is a powerful machine learning algorithm capable of modeling complex non-linear relationships. In this project, SVR was implemented using a Radial Basis Function (RBF) kernel, which allows the model to capture non-

linear patterns in the data.

- Since SVR is sensitive to the scale of data, feature scaling was applied using StandardScaler for both input features and the target variable. This ensures that all variables contribute equally to the model and prevents bias toward features with larger magnitudes.
- The model was trained with parameters such as a high regularization constant (C = 10000) and epsilon = 0.1, allowing it to fit the training data more closely. While SVR has strong predictive capabilities, it can be computationally expensive and may require careful parameter tuning.

4. Decision Tree Regressor

- The Decision Tree Regressor is a non-linear model that splits the dataset into smaller subsets based on feature values. It builds a tree-like structure where each node represents a decision rule, and each leaf node represents a predicted value.
- This model is capable of capturing complex relationships without requiring feature scaling or transformation. It is also easy to interpret, as the decision-making process can be visualized.
- However, decision trees are prone to overfitting, especially when not properly controlled using parameters such as maximum depth or minimum samples per leaf. Despite this limitation, it provides valuable insights into feature importance and decision boundaries.

5. Random Forest Regressor

- Random Forest is an ensemble learning technique that combines multiple decision trees to improve predictive performance and reduce overfitting. Each tree is trained on a random subset of the data, and the final prediction is obtained by averaging the outputs of all trees.

- This model is robust and performs well on complex datasets with high variability. It also provides feature importance scores, which help in understanding the influence of different features on house prices.

- Compared to a single decision tree, Random Forest offers better generalization and stability, making it a strong candidate for regression tasks.

4. Evaluation metrics used

- In this project, multiple evaluation metrics were used to assess the performance of regression models in predicting house prices. Since no single metric can fully capture model effectiveness, a combination of statistical measures and visual analysis techniques was employed to ensure a comprehensive evaluation.

1. R^2 Score (Coefficient of Determination)

- The R^2 score was one of the primary metrics used to evaluate model performance. It represents the proportion of variance in the dependent variable (house price) that is explained by the independent variables.
- An R^2 score closer to 1 indicates a better fit, while a value closer to 0 suggests poor predictive capability.
- Particular emphasis was placed on comparing **training and testing R^2 scores**:
- A high training R^2 with a significantly lower testing R^2 indicates **overfitting**
- Similar values for both suggest **good generalization**
- This comparison played a crucial role in selecting the most reliable model.

2. Mean Absolute Error (MAE)

- Mean Absolute Error measures the average absolute difference between actual and predicted values. It provides a clear interpretation of how far predictions deviate from real values in real-world units (price).
- MAE was used for both training and testing datasets to:
- Evaluate prediction accuracy
- Identify consistency across datasets
- Lower MAE values indicate better model performance and more accurate predictions.

3. Mean Squared Error (MSE) and Root Mean Squared Error (RMSE)

- MSE calculates the average of squared differences between predicted and actual values, giving higher weight to larger errors. RMSE, being the square root of MSE, brings the error back to the original unit scale.
- These metrics were particularly useful for:
- Penalizing large prediction errors
- Identifying models that may perform well on average but fail on extreme values
- RMSE was preferred for interpretation due to its direct relation to the target variable.

4. Train-Test Performance Comparison

A significant focus of the evaluation process was placed on comparing model performance across training and testing datasets. This helped in identifying:

- Overfitting (model memorizes training data)
- Underfitting (model fails to learn patterns)

To further strengthen this analysis, the final selected model was evaluated across multiple train-test split ratios:

- 60:40
- 70:30
- 80:20

This ensured that the model's performance remained stable and consistent across different data distributions.

5. Visual Evaluation Techniques

In addition to numerical metrics, extensive use of graphical representations was made to better understand model behavior.

a) Actual vs Predicted Plot

This scatter plot compares true values with predicted values. A well-performing model shows points clustered closely around the diagonal line.

b) Model Comparison Graphs

Multiple graphs were generated to compare performance across different models. These visualizations helped in:

- Quickly identifying the best-performing model
- Understanding trade-offs between accuracy and error

c) Split Ratio Comparison Visuals

Graphical comparisons across different train-test splits provided insights into model robustness and stability.

5. Preliminary Results and Visualizations

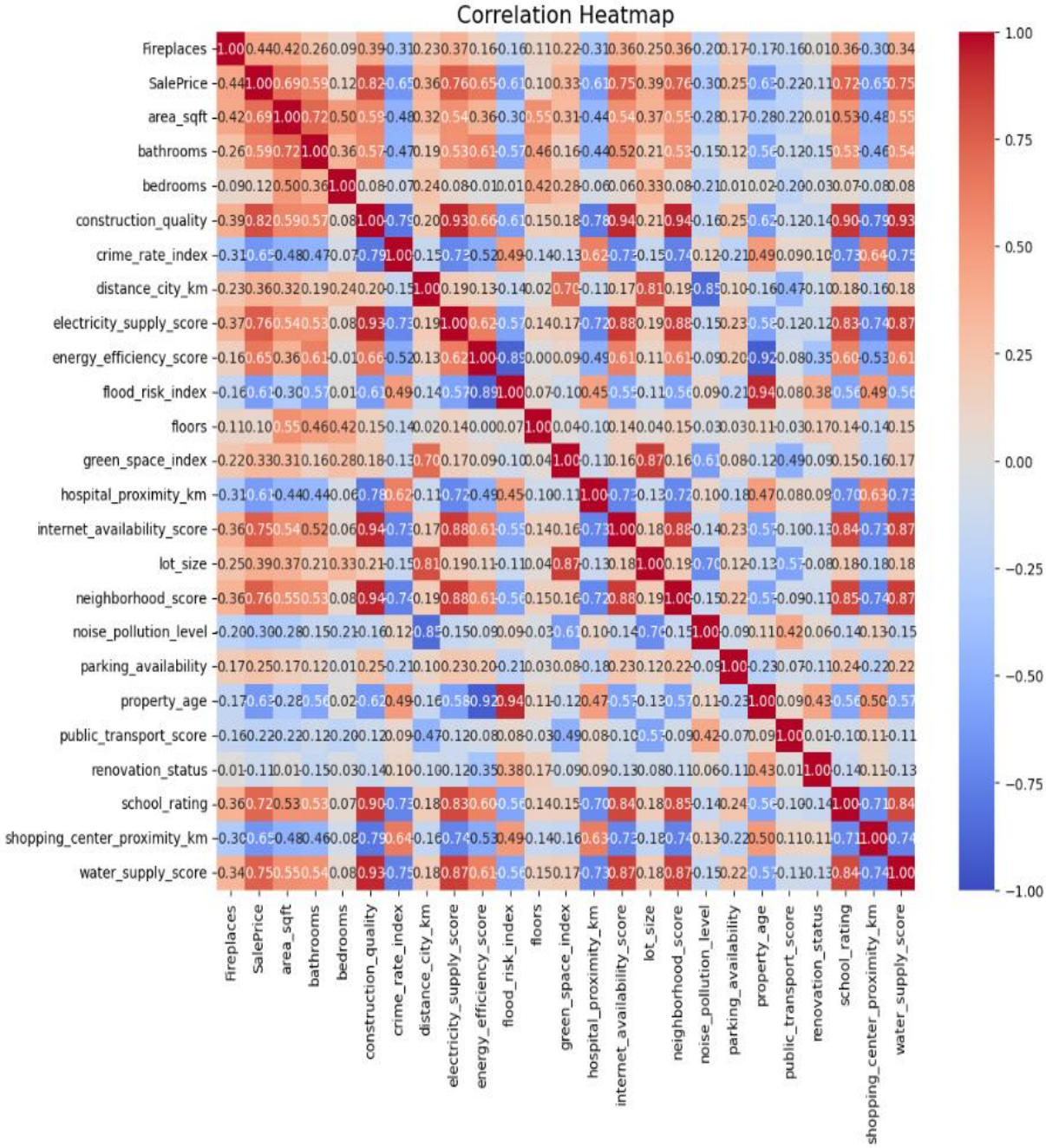


Fig.2 Heatmap showing corelation.

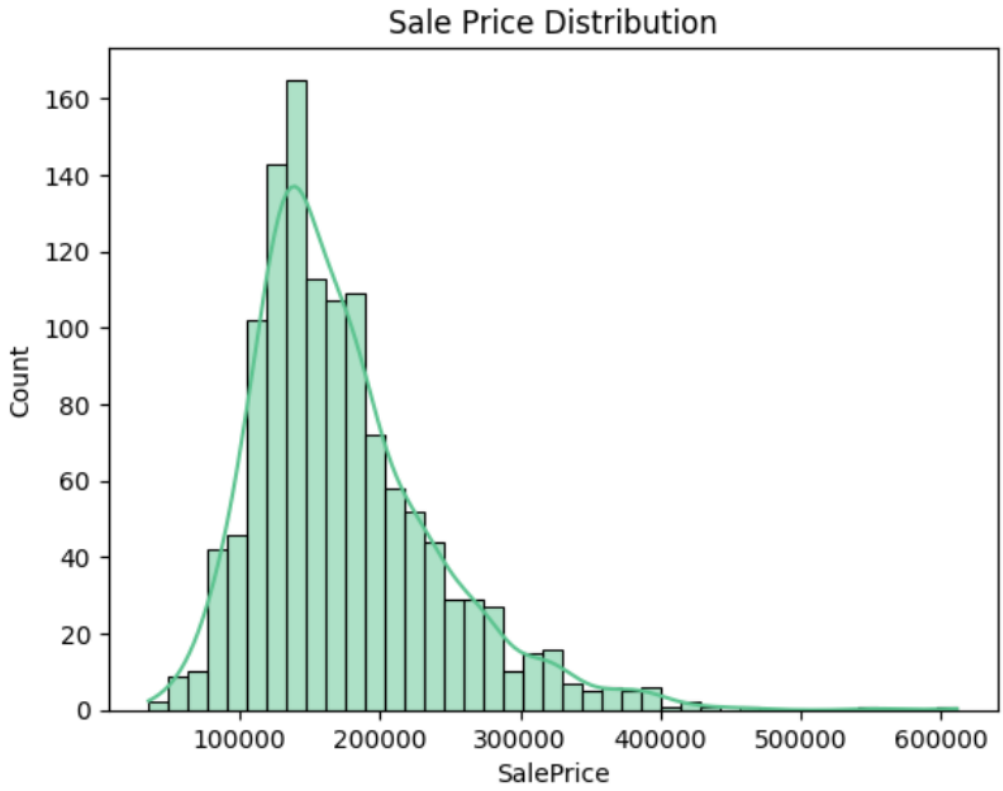


Fig 3: Data Distribution

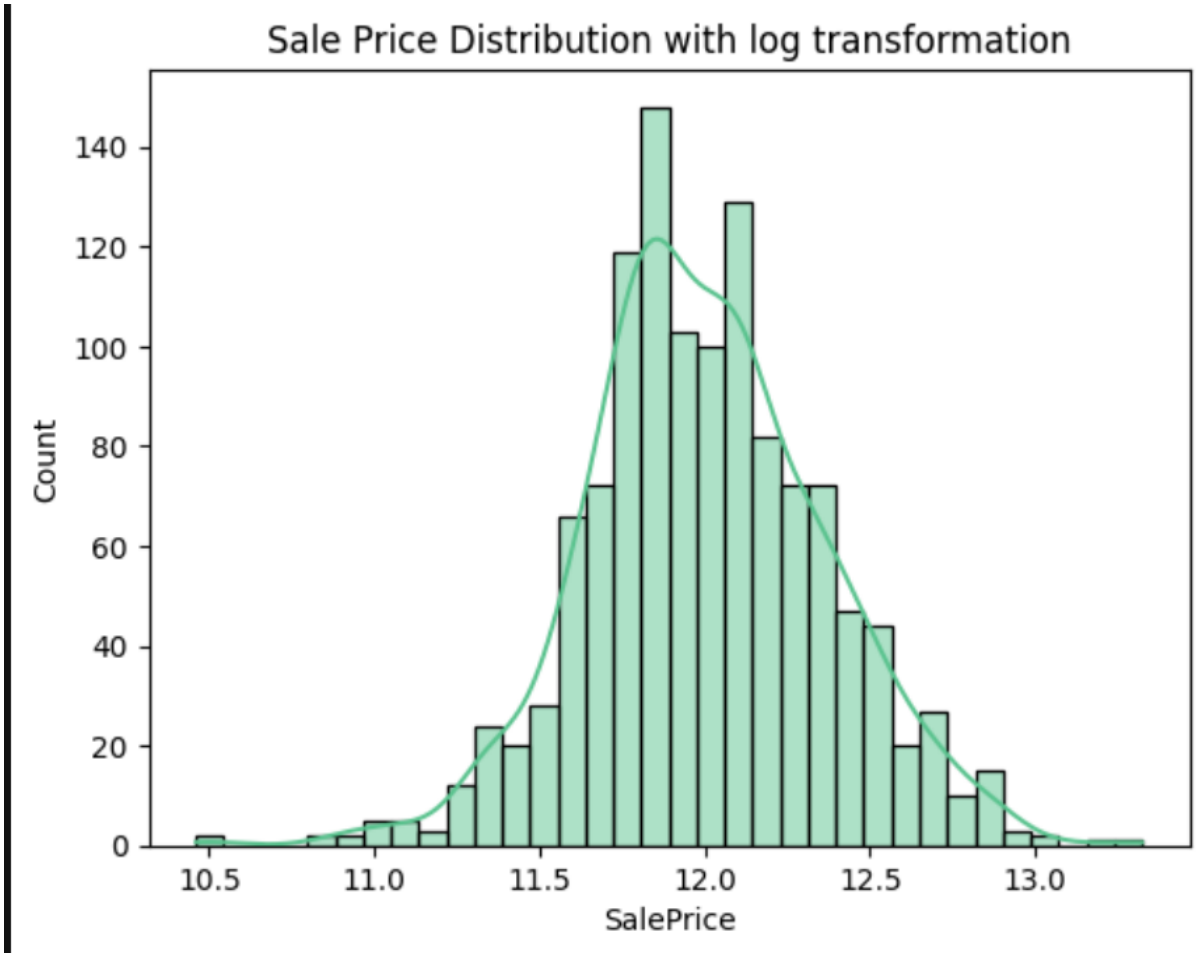


Fig 4: Best fit Line.

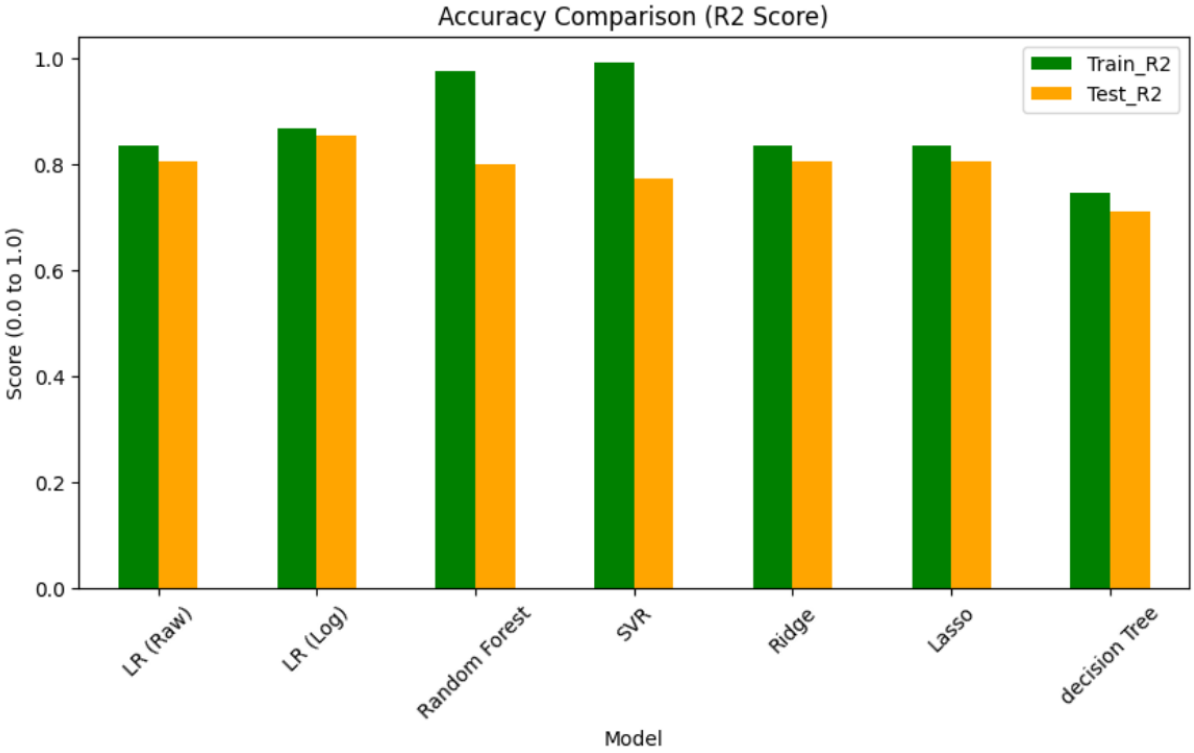


Fig 5: Comparison of different Models

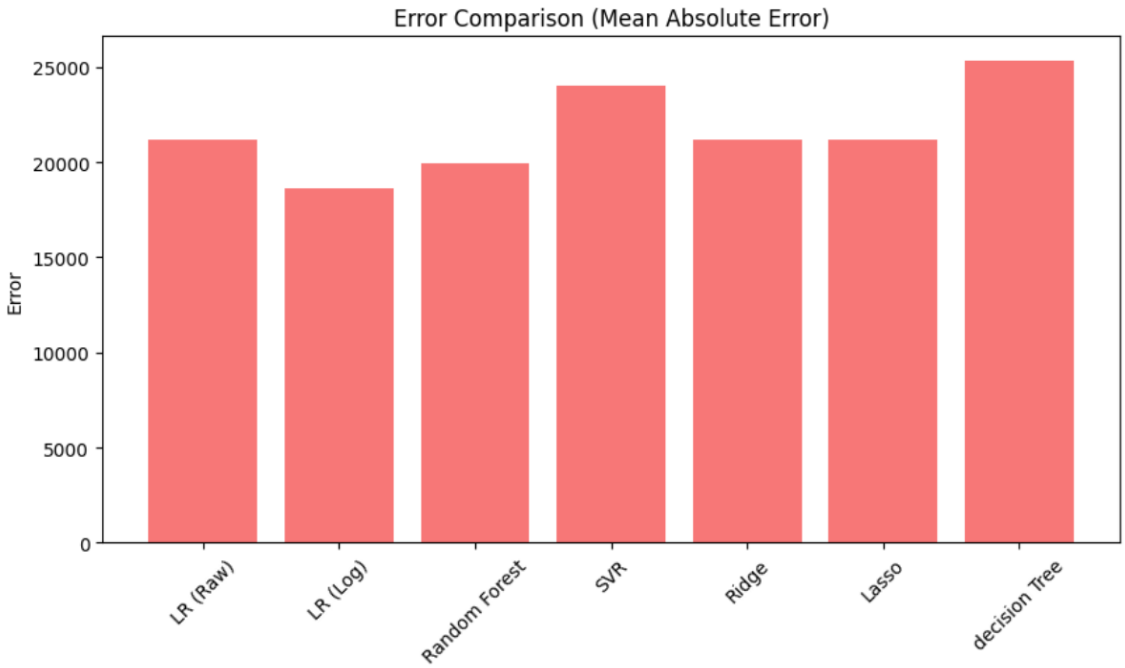


Fig 6: Error Comparison.

6. References

Books:

1. *Let Us Python* – Yashavant Kanetkar

Online Resources:

- GeeksforGeeks – https://www.geeksforgeeks.org/
- TutorialsPoint – https://www.tutorialspoint.com/python/index.htm
- Stack Overflow – https://stackoverflow.com
- Kaggle – https://www.kaggle.com/
- ChatGPT – https://www.chatgpt.com